

Modulo 2: Normalización de un Sistema de Ventas

Karem Juliet Barandica Arias

Tecnólogo en Análisis y Desarrollo de Software

Jeysson Contreras

Servicio Nacional de Aprendizaje (SENA)

Centro de Gestión de Mercados, Logística y Tecnologías de la Información

19 de junio de 2026

Paso 1. Diagnóstico de la tabla cruda

- **Identifica atributos no atómicos.**

Los siguientes campos contienen múltiples valores separados por comas:

productos_codigos
productos_nombres
categorias
cantidades
precios_unitarios
descuentos

- **Marca datos repetidos por cliente, vendedor, producto, categoría y pago.**

Cliente

CC101 | María Gómez

Aparece en:

V1001

V1003

Vendedor

VEN01 | Ana Torres

Aparece en:

V1001

V1003

Productos

P001 Mouse USB

P002 Teclado mecánico

P003 Monitor 24

Categorías

Periféricos

Pantallas

Computadores

Accesorios

Métodos de pago

Transferencia
Tarjeta crédito
Efectivo

- **Explica al menos tres anomalías: inserción, actualización y eliminación.**

Inserción

No es posible registrar un producto nuevo sin crear una venta.

Ejemplo:

P007 Impresora

No puede almacenarse independientemente.

Actualización

Si cambia el correo de María Gómez:

maria.gomez@mail.com

Debe modificarse en todas las filas donde aparezca.

Si una fila no se actualiza:

inconsistencia de datos

Eliminación

Si se elimina la venta:

V1004

también se pierde información de:

Laura Rojas

Diana Mora

Webcam HD

aunque esos datos sigan siendo válidos.

- **Redacta una lista inicial de dependencias funcionales**

venta_id → fecha_venta,

cliente_doc,

vendedor_id,

metodo_pago,

entidad_pago,

total_venta

cliente_doc → cliente_nombre,

cliente_email,

cliente_telefono,

cliente_direccion,

cliente_ciudad
vendedor_id → vendedor_nombre,
vendedor_zona
producto_codigo → producto_nombre,
categoria,
precio_unitario
categoria → nombre_categoria

Paso 2. Aplicacion de 1FN

- **Transforma los campos multivaluados en registros separados**

```
CREATE TABLE ventas_1fn(  
-   venta_id VARCHAR(10),  
-   fecha_venta DATE,  
-   cliente_doc VARCHAR(20),  
-   cliente_nombre VARCHAR(100),  
-   cliente_email VARCHAR(120),  
-   cliente_telefono VARCHAR(30),  
-   cliente_direccion VARCHAR(150),  
-   cliente_ciudad VARCHAR(80),  
-   vendedor_id VARCHAR(10),  
-   vendedor_nombre VARCHAR(100),  
-   vendedor_zona VARCHAR(80),  
-   producto_codigo VARCHAR(10),  
-   producto_nombre VARCHAR(100),  
-   categoria VARCHAR(100),  
-   cantidad int,  
-   precio_unitario NUMERIC(12,2),  
-   descuento NUMERIC(12,2),  
-   metodo_pago VARCHAR(80),  
-   entidad_pago VARCHAR(80),  
-   total_venta NUMERIC(12,2)  
- );
```

- **Cada fila debe representar una venta con un solo producto asociado**

```
INSERT INTO ventas_crudas VALUES  
(venta_id, fecha_venta, cliente_doc, cliente_nombre, cliente_email,  
cliente_telefono, cliente_direccion, cliente_ciudad, vendedor_id,  
vendedor_nombre, vendedor_zona, producto_codigo, producto_nombre,  
categoría, cantidad, precio_unitario, descuento, metodo_pago,  
entidad_pago, total_venta)
```

('V1001','2026-04-01','CC101','Maria
Gomez','maria.gomez@mail.com','3101112233',
'Calle 10 # 5-20','Bogota','VEN01','Ana Torres','Norte',
'P001','Mouse USB','Perifericos', 2,45000,0,'Transferencia',
'Bancolombia',90000),

('V1001','2026-04-01','CC101','Maria
Gomez','maria.gomez@mail.com','3101112233', 0
'Calle 10 # 5-20','Bogota','VEN01','Ana Torres','Norte',
'P002','Teclado mecanico','Perifericos',
1,180000,0,'Transferencia','Bancolombia',180000),

('V1002','2026-04-02','CC102','Juan
Perez','juan.perez@mail.com','3155558899',
'Carrera 8 # 20-15','Bogota','VEN02','Carlos Ruiz','Centro',
'P003','Monitor 24','Pantallas',
1,720000,20000,'Tarjeta credito','Visa',700000),

('V1002','2026-04-02','CC102','Juan
Perez','juan.perez@mail.com','3155558899',
'Carrera 8 # 20-15','Bogota','VEN02','Carlos Ruiz','Centro',
'P001','Mouse USB','Perifericos',
1,45000,0,'Tarjeta credito','Visa',45000),

('V1003','2026-04-03','CC101','Maria
Gomez','maria.gomez@mail.com','3101112233',
'Calle 10 # 5-20','Bogota','VEN01','Ana Torres','Norte',
'P004','Portatil 14', 'Computadores',1,2450000,
50000,'Transferencia','Bancolombia',2400000),

('V1003','2026-04-03','CC101','Maria
Gomez','maria.gomez@mail.com','3101112233',
'Calle 10 # 5-20','Bogota','VEN01','Ana Torres','Norte',
'P002','Teclado mecanico', 'Perifericos',2,180000,
0,'Transferencia','Bancolombia',360000),

('V1003','2026-04-03','CC101','Maria
Gomez','maria.gomez@mail.com','3101112233',
'Calle 10 # 5-20','Bogota','VEN01','Ana Torres','Norte',
'P005','Base refrigerante', 'Accesorios', 1, 95000,
0,'Transferencia','Bancolombia',95000),

```

('V1004','2026-04-04','CC103','Laura
Rojas','laura.rojas@mail.com','3209994455',
'Av. 68 # 45-30','Medellin','VEN03','Diana Mora','Occidente',
P003,'Monitor 24','Pantallas',
2,720000,0,'Efectivo','Caja principal',1440000);
('V1004','2026-04-04','CC103','Laura
Rojas','laura.rojas@mail.com','3209994455',
'Av. 68 # 45-30','Medellin','VEN03','Diana Mora','Occidente',
'P006','Webcam HD','Perifericos',
1,150000,10000,'Efectivo','Caja principal',140000);

```

- **Propone una clave compuesta provisional para la tabla en 1FN.**

Clave provisional:

(venta_id, producto_codigo)

Paso 3. Aplicación de 2FN

- **Evalúa si la tabla en 1FN usa una clave compuesta.**
- **Separa los atributos que dependen solo de una parte de la clave.**
- **Identifica entidades como venta, cliente, vendedor, producto y detalle de venta.**
- **Define claves primarias y foráneas para las entidades resultantes.**

```

CREATE TABLE clientes (
cliente_doc VARCHAR(20) PRIMARY KEY,
cliente_nombre VARCHAR(100) NOT NULL,
cliente_email VARCHAR(30) NOT NULL,
cliente_telefono VARCHAR(100) NOT NULL,
cliente_direccion VARCHAR(150) NOT NULL,
cliente_ciudad VARCHAR(80) NOT NULL
);
CREATE TABLE vendedores(
vendedor_id VARCHAR(10) PRIMARY KEY,
vendedor_nombre VARCHAR(100) NOT NULL,
vendedor_zona VARCHAR(100) NOT NULL
);
CREATE TABLE productos(
producto_codigo VARCHAR(10) PRIMARY KEY,

```

```

producto_nombre VARCHAR(100),
categoria VARCHAR(100)
);
CREATE TABLE ventas(
venta_id VARCHAR(10) PRIMARY KEY,
fecha_venta DATE,
cliente_doc VARCHAR(20),
vendedor_id VARCHAR(10),
metodo_pago VARCHAR(80),
entidad_pago VARCHAR(80),
CONSTRAINT fk_clientes
FOREIGN KEY (cliente_doc) REFERENCES clientes(cliente_doc),
CONSTRAINT fk_vendedores
FOREIGN KEY (vendedor_id) REFERENCES vendedores(vendedor_id)
);
CREATE TABLE detalle_ventas(
venta_id VARCHAR(10),
producto_codigo VARCHAR(10),
cantidad INT,
precio_unitario NUMERIC(12,2),
descuento NUMERIC(12,2),
PRIMARY KEY (venta_id, producto_codigo),
CONSTRAINT fk_ventas
FOREIGN KEY (venta_id) REFERENCES ventas(venta_id),
CONSTRAINT fk_productos
FOREIGN KEY (producto_codigo) REFERENCES productos(producto_codigo)
);

```

Paso 4. Aplicación de 3FN

- **Elimina dependencias transitivas.**
- **Separa categorías de productos si la categoría depende del producto y no de la venta.**
- **Separa ciudades o zonas solo si justificas que se administraran como catálogos.**
- **Revisa que cada atributo dependa de la clave, de toda la clave y de nada más que la clave.**

```

CREATE TABLE departamentos (
    id_departamento VARCHAR(10) PRIMARY KEY,
    nombre VARCHAR(100)
);

CREATE TABLE ciudades (
    id_ciudad VARCHAR(10) PRIMARY KEY,
    nombre VARCHAR(100),
    id_departamento VARCHAR(10),
    CONSTRAINT fk_departamentos
    FOREIGN KEY (id_departamento)
    REFERENCES departamentos(id_departamento)
);

CREATE TABLE clientes (
    cliente_doc VARCHAR(20) PRIMARY KEY,
    cliente_nombre VARCHAR(100) NOT NULL,
    cliente_email VARCHAR(30) NOT NULL,
    cliente_telefono VARCHAR(100) NOT NULL,
    cliente_direccion VARCHAR(150) NOT NULL,
    id_ciudad VARCHAR(10) NOT NULL,
    CONSTRAINT fk_ciudades
    FOREIGN KEY (id_ciudad) REFERENCES ciudades(id_ciudad)
);

CREATE TABLE zonas(
    id_zona VARCHAR(10) PRIMARY KEY,
    zona_nombre VARCHAR(100)
);

CREATE TABLE vendedores(
    vendedor_id VARCHAR(10) PRIMARY KEY,
    vendedor_nombre VARCHAR(100) NOT NULL,
    id_zona VARCHAR(10) NOT NULL,
    CONSTRAINT fk_zonas
    FOREIGN KEY (id_zona) REFERENCES zonas(id_zona)
);

CREATE TABLE categorias(
    id_categoria VARCHAR(10) PRIMARY KEY,
    categoria_nombre VARCHAR(100)
);

```

```

CREATE TABLE productos(
producto_codigo VARCHAR(10) PRIMARY KEY,
producto_nombre VARCHAR(100),
id_categoria VARCHAR(10),
CONSTRAINT fk_categorias
FOREIGN KEY (id_categoria) REFERENCES categorias (id_categoria)
);
CREATE TABLE metodos_pago(
id_metodo VARCHAR(10) PRIMARY KEY,
metodo_nombre VARCHAR(80)
);
CREATE TABLE entidades_pago(
id_entidad VARCHAR(10) PRIMARY KEY,
entidad_nombre VARCHAR(80)
);
CREATE TABLE ventas(
venta_id VARCHAR(10) PRIMARY KEY,
fecha_venta DATE,
cliente_doc VARCHAR(20),
vendedor_id VARCHAR(10),
id_metodo VARCHAR(10),
id_entidad VARCHAR(10),
FOREIGN KEY (id_entidad) REFERENCES entidades_pago(id_entidad),
FOREIGN KEY (id_metodo) REFERENCES metodos_pago(id_metodo),
CONSTRAINT fk_clientes
FOREIGN KEY (cliente_doc) REFERENCES clientes(cliente_doc),
CONSTRAINT fk_vendedores
FOREIGN KEY (vendedor_id) REFERENCES vendedores(vendedor_id)
);
CREATE TABLE detalle_ventas(
venta_id VARCHAR(10),
producto_codigo VARCHAR(10),
cantidad INT,
precio_unitario NUMERIC(12,2),
descuento NUMERIC(12,2),
PRIMARY KEY (venta_id, producto_codigo),
CONSTRAINT fk_ventas
FOREIGN KEY (venta_id) REFERENCES ventas(venta_id),

```

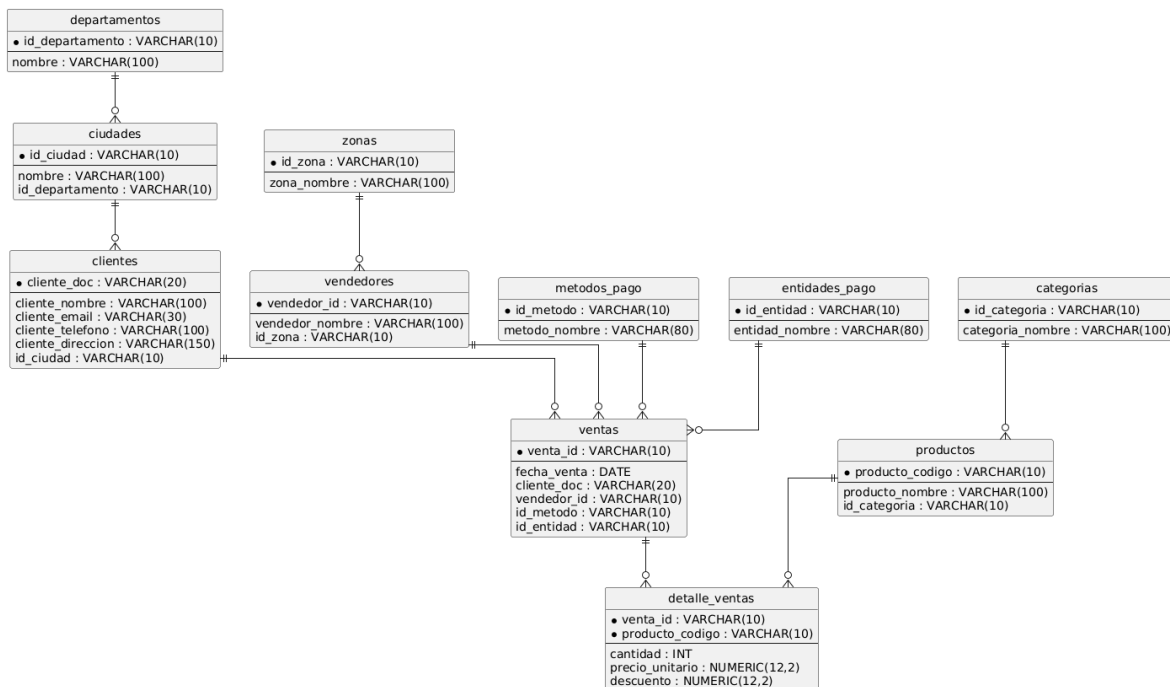
```

CONSTRAINT fk_productos
FOREIGN KEY (producto_codigo) REFERENCES productos(producto_codigo)
);

```

Paso 5. Diseño del modelo entidad-relación

- **Construye el diagrama ER con entidades, atributos, claves y cardinalidades.**
- **Incluye como mínimo: clientes, vendedores, ventas, detalle_venta, productos y categorías.**
- **Puedes agregar metodos_pago o ciudades si tu justificación lo requiere.**
- **No uses la tabla ventas_crudas como tabla transaccional final.**



Paso 6. Implementación física en PostgreSQL

- **Crea el esquema normalizado con CREATE TABLE.**
- **Agrega PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE y CHECK cuando corresponda.**
- **Inserta los datos normalizados derivados de la tabla cruda.**
- **Evita duplicar clientes, vendedores, categorías y productos.**

```
CREATE SCHEMA IF NOT EXISTS modulo2_normalizacion;

CREATE TABLE modulo2_normalizacion.departamentos(
    id_departamento VARCHAR(10) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE modulo2_normalizacion.ciudades(
    id_ciudad VARCHAR(10) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    id_departamento VARCHAR(10) NOT NULL,
    CONSTRAINT fk_departamentos
        FOREIGN KEY (id_departamento) REFERENCES
modulo2_normalizacion.departamentos(id_departamento),
    CONSTRAINT uq_ciudad_departamento
        UNIQUE(nombre, id_departamento)
);

CREATE TABLE modulo2_normalizacion.clientes(
    cliente_doc VARCHAR(20) PRIMARY KEY,
    cliente_nombre VARCHAR(100) NOT NULL,
    cliente_email VARCHAR(100) NOT NULL UNIQUE,
    cliente_telefono VARCHAR(20) NOT NULL,
    cliente_direccion VARCHAR(150) NOT NULL,
    id_ciudad VARCHAR(10) NOT NULL,
    CONSTRAINT fk_ciudades
        FOREIGN KEY (id_ciudad)
        REFERENCES modulo2_normalizacion.ciudades(id_ciudad)
);

CREATE TABLE modulo2_normalizacion.zonas(
    id_zona VARCHAR(10) PRIMARY KEY,
    zona_nombre VARCHAR(100) NOT NULL UNIQUE
);
```

```

CREATE TABLE modulo2_normalizacion.vendedores(
    vendedor_id VARCHAR(10) PRIMARY KEY,
    vendedor_nombre VARCHAR(100) NOT NULL,
    id_zona VARCHAR(10) NOT NULL,
    CONSTRAINT fk_zonas
        FOREIGN KEY (id_zona)
        REFERENCES modulo2_normalizacion.zonas(id_zona)
);

CREATE TABLE modulo2_normalizacion.categorias(
    id_categoria VARCHAR(10) PRIMARY KEY,
    categoria_nombre VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE modulo2_normalizacion.productos(
    producto_codigo VARCHAR(10) PRIMARY KEY,
    producto_nombre VARCHAR(100) NOT NULL,
    id_categoria VARCHAR(10) NOT NULL,
    CONSTRAINT fk_categorias
        FOREIGN KEY (id_categoria)
        REFERENCES modulo2_normalizacion.categorias(id_categoria)
);

CREATE TABLE modulo2_normalizacion.metodos_pago(
    id_metodo VARCHAR(10) PRIMARY KEY,
    metodo_nombre VARCHAR(80) NOT NULL UNIQUE
);

CREATE TABLE modulo2_normalizacion.entidades_pago(
    id_entidad VARCHAR(10) PRIMARY KEY,
    entidad_nombre VARCHAR(80) NOT NULL UNIQUE
);

CREATE TABLE modulo2_normalizacion.ventas(

```

```

venta_id VARCHAR(10) PRIMARY KEY,

fecha_venta DATE NOT NULL,

cliente_doc VARCHAR(20) NOT NULL,

vendedor_id VARCHAR(10) NOT NULL,

id_metodo VARCHAR(10) NOT NULL,

id_entidad VARCHAR(10) NOT NULL,

CONSTRAINT fk_clientes

    FOREIGN KEY (cliente_doc)

    REFERENCES modulo2_normalizacion.clientes(cliente_doc),

CONSTRAINT fk_vendedores

    FOREIGN KEY (vendedor_id)

    REFERENCES modulo2_normalizacion.vendedores(vendedor_id),

CONSTRAINT fk_metodos_pago

    FOREIGN KEY (id_metodo)

    REFERENCES modulo2_normalizacion.metodos_pago(id_metodo),

CONSTRAINT fk_entidades_pago

    FOREIGN KEY (id_entidad)

    REFERENCES modulo2_normalizacion.entidades_pago(id_entidad)

);

CREATE TABLE modulo2_normalizacion.detalle_ventas(

    venta_id VARCHAR(10),

    producto_codigo VARCHAR(10),

    cantidad INT NOT NULL,

    precio_unitario NUMERIC(12,2) NOT NULL,

    descuento NUMERIC(12,2) DEFAULT 0,

    PRIMARY KEY (venta_id, producto_codigo),

    CONSTRAINT fk_ventas

        FOREIGN KEY (venta_id)

        REFERENCES modulo2_normalizacion.ventas(venta_id),

```

```

CONSTRAINT fk_productos
    FOREIGN KEY (producto_codigo)
    REFERENCES modulo2_normalizacion.productos(producto_codigo),
CONSTRAINT chk_cantidad
    CHECK (cantidad > 0),
CONSTRAINT chk_precio
    CHECK (precio_unitario > 0),
CONSTRAINT chk_descuento
    CHECK (descuento >= 0)
);

INSERT INTO modulo2_normalizacion.departamentos (id_departamento, nombre) VALUES
('DEP01','Cundinamarca'),
('DEP02','Antioquia');

INSERT INTO modulo2_normalizacion.ciudades (id_ciudad, nombre, id_departamento) VALUES
('CIU01','Bogota','DEP01'),
('CIU02','Medellin','DEP02');

INSERT INTO modulo2_normalizacion.clientes (cliente_doc, cliente_nombre, cliente_email,
cliente_telefono, cliente_direccion, id_ciudad ) VALUES
('CC101','Maria Gomez','maria.gomez@mail.com','3101112233','Calle 10 # 5-20','CIU01'),
('CC102','Juan Perez','juan.perez@mail.com','3155558899','Carrera 8 # 20-15','CIU01'),
('CC103','Laura Rojas','laura.rojas@mail.com','3209994455','Av. 68 # 45-30','CIU02');

INSERT INTO modulo2_normalizacion.zonas (id_zona, zona_nombre) VALUES
('Z01','Norte'),
('Z02','Centro'),
('Z03','Occidente');

INSERT INTO modulo2_normalizacion.vendedores (vendedor_id, vendedor_nombre, id_zona) VALUES
('VEN01','Ana Torres','Z01'),
('VEN02','Carlos Ruiz','Z02'),
('VEN03','Diana Mora','Z03');

INSERT INTO modulo2_normalizacion.categorias (id_categoria, categoria_nombre) VALUES

```

```

('CAT01','Perifericos'),

('CAT02','Pantallas'),

('CAT03','Computadores'),

('CAT04','Accesorios');

INSERT INTO modulo2_normalizacion.productos (producto_codigo, producto_nombre, id_categoria)
VALUES

('P001','Mouse USB','CAT01'),

('P002','Teclado mecanico','CAT01'),

('P003','Monitor 24','CAT02'),

('P004','Portatil 14','CAT03'),

('P005','Base refrigerante','CAT04'),

('P006','Webcam HD','CAT01');

INSERT INTO modulo2_normalizacion.metodos_pago (id_metodo, metodo_nombre) VALUES

('MP01','Transferencia'),

('MP02','Tarjeta credito'),

('MP03','Efectivo');

INSERT INTO modulo2_normalizacion.entidades_pago (id_entidad, entidad_nombre) VALUES

('EP01','Bancolombia'),

('EP02','Visa'),

('EP03','Caja principal');

INSERT INTO modulo2_normalizacion.ventas (venta_id, fecha_venta, cliente_doc, vendedor_id,
id_metodo, id_entidad) VALUES

('V1001','2026-04-01','CC101','VEN01','MP01','EP01'),

('V1002','2026-04-02','CC102','VEN02','MP02','EP02'),

('V1003','2026-04-03','CC101','VEN01','MP01','EP01'),

('V1004','2026-04-04','CC103','VEN03','MP03','EP03');

INSERT INTO modulo2_normalizacion.detalle_ventas (venta_id, producto_codigo, cantidad,
precio_unitario, descuento) VALUES

('V1001','P001',2,45000,0),

('V1001','P002',1,180000,0),

```

```

('V1002','P003',1,720000,20000),
('V1002','P001',1,45000,0),
('V1003','P004',1,2450000,50000),
('V1003','P002',2,180000,0),
('V1003','P005',1,95000,0),
('V1004','P003',2,720000,0),
('V1004','P006',1,150000,10000);

```

Paso 7. Validación con consultas SQL

- **Reconstruye el total de cada venta desde el detalle.**
- **Consulta productos más vendidos por cantidad.**
- **Consulta ventas por vendedor.**
- **Consulta historial de compras por cliente.**
- **Verifica que no existan registros huérfanos en detalle_venta.**

-- TOTAL VENTA --

```

SELECT
    v.venta_id,
    v.fecha_venta,
    c.cliente_nombre,
    SUM((dv.cantidad * dv.precio_unitario) - dv.descuento) AS total_venta
FROM modulo2_normalizacion.ventas v
INNER JOIN modulo2_normalizacion.clientes c
    ON v.cliente_doc = c.cliente_doc
INNER JOIN modulo2_normalizacion.detalle_ventas dv
    ON v.venta_id = dv.venta_id
GROUP BY
    v.venta_id,
    v.fecha_venta,
    c.cliente_nombre
ORDER BY v.venta_id;

```

-- PRODUCTOS MÁS VENDIDOS POR CANTIDAD --

```
SELECT

    p.producto_codigo,

    p.producto_nombre,

    SUM(dv.cantidad) AS total_vendido

FROM modulo2_normalizacion.productos p

INNER JOIN modulo2_normalizacion.detalle_ventas dv

    ON p.producto_codigo = dv.producto_codigo

GROUP BY

    p.producto_codigo,

    p.producto_nombre

ORDER BY total_vendido DESC;
```

-- VENTAS POR VENDEDOR--

```
SELECT

    vd.vendedor_id,

    vd.vendedor_nombre,

    COUNT(v.venta_id) AS numero_ventas

FROM modulo2_normalizacion.vendedores vd

INNER JOIN modulo2_normalizacion.ventas v

    ON vd.vendedor_id = v.vendedor_id

GROUP BY

    vd.vendedor_id,

    vd.vendedor_nombre

ORDER BY numero_ventas DESC;
```

-- VENTAS TOTALES POR VENDEDOR --

```
SELECT

    vd.vendedor_nombre,

    SUM((dv.cantidad * dv.precio_unitario) - dv.descuento) AS total_vendido

FROM modulo2_normalizacion.vendedores vd

INNER JOIN modulo2_normalizacion.ventas v
```

```

    ON vd.vendedor_id = v.vendedor_id

INNER JOIN modulo2_normalizacion.detalle_ventas dv

    ON v.venta_id = dv.venta_id

GROUP BY vd.vendedor_nombre

ORDER BY total_vendido DESC;

-- HISTORIAL DE COMPRAS POR CLIENTE --

SELECT

    c.cliente_doc,

    c.cliente_nombre,

    v.venta_id,

    v.fecha_venta,

    SUM((dv.cantidad * dv.precio_unitario) - dv.descuento) AS total_compra

FROM modulo2_normalizacion.clientes c

INNER JOIN modulo2_normalizacion.ventas v

    ON c.cliente_doc = v.cliente_doc

INNER JOIN modulo2_normalizacion.detalle_ventas dv

    ON v.venta_id = dv.venta_id

GROUP BY

    c.cliente_doc,

    c.cliente_nombre,

    v.venta_id,

    v.fecha_venta

ORDER BY

    c.cliente_nombre,

    v.fecha_venta;

-- VERIFICAR REGISTROS HUÉRFANOS EN DETALLE_VENTAS --

SELECT dv.*

FROM modulo2_normalizacion.detalle_ventas dv

LEFT JOIN modulo2_normalizacion.productos p

```

```

ON dv.producto_codigo = p.producto_codigo
WHERE p.producto_codigo IS NULL;

```

-- VENTAS INEXISTENTES--

```

SELECT dv.*
FROM modulo2_normalizacion.detalle_ventas dv
LEFT JOIN modulo2_normalizacion.ventas v
ON dv.venta_id = v.venta_id
WHERE v.venta_id IS NULL;

```

7. Plantilla de analisis de dependencias funcionales

Completa esta tabla en tu informe antes de construir el modelo final:

Dependencia funcional	Tipo	Acción de normalizacion
cliente_doc -> cliente_nombre, cliente_email, cliente_telefono, cliente_direccion	Dependencia funcional completa	Crear la tabla clientes con cliente_doc como clave primaria.
vendedor_id -> vendedor_nombre, vendedor_zona	Dependencia funcional completa	Crear la tabla vendedores y separar las zonas en una tabla zonas para eliminar redundancia.
producto_codigo -> producto_nombre, categoria	Dependencia funcional completa	Crear la tabla productos y separar las categorías en la tabla categorias.
venta_id -> fecha_venta, cliente_doc, vendedor_id, metodo_pag	Dependencia funcional completa	Crear la tabla ventas y catalogar los métodos y entidades de pago en tablas independientes.
venta_id + producto_codigo -> cantidad, precio_unitario, descuento	Dependencia funcional completa con clave compuesta	Crear la tabla detalle_ventas con clave primaria compuesta (venta_id, producto_codigo).